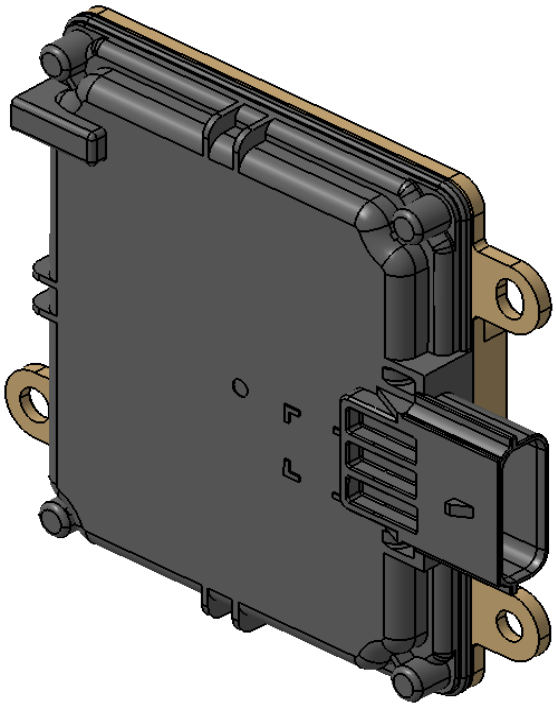
Technical Customer Information

FLR26 Software Reprogramming Guide



Copyright 2024 © Knorr-Bremse AG. All rights reserved, including industrial property rights applications.
Knorr-Bremse AG retains any power of disposal, such as for copying and transferring.

Modifications

Rev.	Date	Author	Section	Modification	Checked by
000	2024-05-06	Ren, Bo	All	Initial Version	Zhan, Zhuyi

Copyright 2024 © Knorr-Bremse AG. All rights reserved, including industrial property rights applications.
Knorr-Bremse AG retains any power of disposal, such as for copying and transferring.

Table of Content

1 Scope 4

2 Description 4

3 Overview of Download and Written 4

4 Programming Sequence 5

4.1 Entry ExtendedDiagnosticSession 5

4.2 Security Unlock for ExtendedDiagnosticSession 5

4.3 Write Fingerprint 6

4.4 Write Application-Data 7

4.5 Reset ECU 7

5 Specification of UDS Services 8

5.1 (0x10) DiagnosticSessionControl Service 8

5.2 (0x11) ECUReset Service 9

5.3 (0x27) SecurityAccess Service 9

5.4 (0x2E) WriteDataByIdentifier Service 11

6 Appendix – Example Logs 12

Copyright 2024 © Knorr-Bremse AG. All rights reserved, including industrial property rights applications.
Knorr-Bremse AG retains any power of disposal, such as for copying and transferring.

1 Scope

This document describes the Application-Data written process via UDS for FLR26.

2 Description

The Application-Data written process is basically a standard UDS process. This document describes the written sequence for Application-Data. For written unified diagnostics services (UDS) protocol is used to send diagnostic messages on the CAN.

3 Overview of Download and Written

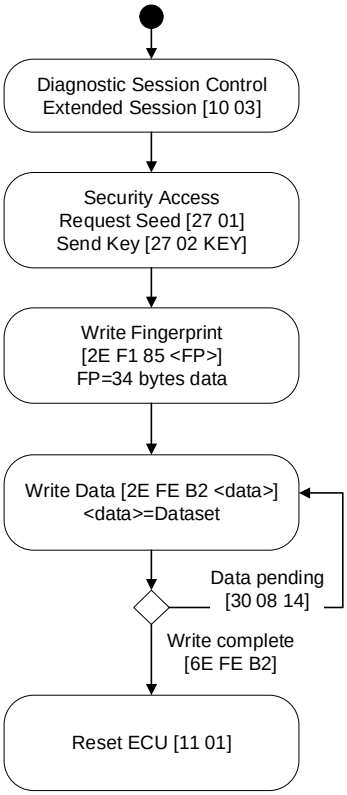
For FLR26 radar, the bootloader software and application software are programmed at supplier side, and cannot be updated by UDS on CAN, only Application-Data written is possible.

The Application-Data written can be done according to the HYDiagFlash_Tool that be provided by KB, or the tool be developed by OEM selves.

Copyright 2024 © Knorr-Bremse AG. All rights reserved, including industrial property rights applications.
Knorr-Bremse AG retains any power of disposal, such as for copying and transferring.

4 Programming Sequence

Steps for reprogramming:



4.1 Entry ExtendedDiagnosticSession

The programming event is started in the server via a physically addressed request of the Diagnostic Session Control (0x10) service with sessionType equal to ExtendedDiagnosticSession. When the server receives the request, it shall allocate all necessary resources required for programming. It is implementation specific whether the server starts executing out of boot memory.

Request: 10 03

Response: 50 03 00 32 01 F4

4.2 Security Unlock for ExtendedDiagnosticSession

The server shall be unsecured to allow server memory to be erased/flashed using diagnostics request. Initially the client shall request the server for the seed (random number generated at server side). Upon receiving the seed from server the client shall send request with the key to the server. Key is calculated by the client using the seed received from the server and running algorithm on it. Meanwhile on sending the seed to the client, the server shall also start calculation of the key using the same sent seed value. On receiving the correct key from client, the server shall be unsecured.

Example:

Copyright 2024 © Knorr-Bremse AG. All rights reserved, including industrial property rights applications.
Knorr-Bremse AG retains any power of disposal, such as for copying and transferring.

a. Request for seed

Request: 27 01

Response: 67 01 A5 5B F4 05

b. Request with key

Request: 27 02 C3 A7 D5 B5

Response: 67 02

4.3 Write Fingerprint

Upon un-securing the server, fingerprint shall be written in the non-volatile memory. Fingerprint shall consist of the information as of when the server was reprogrammed, and which test tool was used to reprogram it. We maintain three fingerprint records at a time in the memory. The request is of 34 bytes. Fingerprint to be written needs to be presented in first 12bytes of the request, other bytes need to be “0” (don’t care).

Write Fingerprint Request format:

Byte No.	Description	Contents	Cvt
#1	NumberOfModules	0x1(always 0x1)	M
#2 . . #12	Application software Fingerprint #1 Programming date (Date on which the Software is flashed) byte #1 byte #2 byte #3 byte #4 TesterSerialNumber (Name/serial number of tester involved in flashing) byte #5 byte #6 byte #7 byte #8 Application identifier (Be used for tester identification) byte #9 Version of the application (Be used for tester version) byte #10 byte #11	 YY YY MM DD XX XX XX XX FF FF FF	 M M M M M M M M M M M
#13 . . #23	Application software Fingerprint #2 See bytes (#2…. #12)		M
#24 . . #34	Application software Fingerprint #3 See bytes (#2…. #12)		M

Copyright 2024 © Knorr-Bremse AG. All rights reserved, including industrial property rights applications.
Knorr-Bremse AG retains any power of disposal, such as for copying and transferring.

Example:

For Application-Data written:

Request: 2E F1 85 01 07 E4 08 1F 4B 42 44 00 FF FF FF 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Response: 6E F1 85

4.4 Write Application-Data

The WriteDataByIdentifier(0x2E) service shall be used to write the dataset software in the non-volatile memory. The dataset software block size shall be 1k (1024 byte).

Request: 2E FE B2 XX XX XX… (1024 BYTE)

Response: 6E FE B2

4.5 Reset ECU

The client transmits an ECUReset (0x11) service request message with resetType 0x01 (hardReset). The request is transmitted physically addressed to the server. The positive response message is required. The reception of the ECUReset (0x11) request message causes the server to perform a reset and to start running the application software in Default Session. After sending the positive response message the server must be able to reestablish communication within 1000ms.

Request: 11 01

Response: 51 01

Copyright 2024 © Knorr-Bremse AG. All rights reserved, including industrial property rights applications.
Knorr-Bremse AG retains any power of disposal, such as for copying and transferring.

5 Specification of UDS Services

5.1 (0x10) DiagnosticSessionControl Service

5.1.1 DiagnosticSessionControl Request

Below table defines the request message. The SuppressPosRspMsgIndicationBit (bit7) is also supported but not shown.

Byte	Parameter Name	Cvt	Value [Hex]	Mnemonic
#1	DiagnosticSessionControl Request SID	M	10	DSC
#2	subFunction : diagnosticSessionType =	M	XX=	DST
	extendedSession		03	

Whenever the service DSC (DiagnosticSessionControl) is called successfully (that includes also calling DSC with the currently active diagnosticSessionType), the securityAccess rights and the timing parameters ($P2_{SERVER_MAX}$, $P2^*_{SERVER_MAX}$, $S3_{SERVER_MAX}$) will be initialized again.

The table below lists the values for the initialized timing parameters in DSC.

sessionParameter	DiagnosticSessionType			
	Default		Extended	
	Unit [ms]	Value [Hex]	Unit [ms]	Value [Hex]
P2CAN_SERVER_MAX (Resolution: 1ms)	50	00 32	50	00 32
P2*CAN_ SERVER_MAX (Resolution: 10ms)	500	01 F4	500	01 F4

Note: If a DiagnosticSessionControl request is used without parameter diagnosticSessionType, it will be answered by the negative response “incorrectMessageLengthOrInvalidFormat” and the current diagnosticSession-Type remains active.

5.1.2 DiagnosticSessionControl Positive Response

Below table defines DiagnosticSessionControl positive response message flow example.

Byte	Parameter Name	Cvt	Value [Hex]	Mnemonic
#1	DiagnosticSessionControl Positive Response	M	50	DSCPR
#2	diagnosticSessionType = extendedSession	M	03	DST
#3 #4 #5 #6	sessionParameterRecord = [$P2_{SERVER_MAX}$ (high byte) $P2_{SERVER_MAX}$ (low byte) $P2^*_{SERVER_MAX}$ (high byte) $P2^*_{SERVER_MAX}$ (low byte)]	M	00 32 01 F4	SPREC

5.1.3 Supported Negative Response Codes

NRC [Hex]	NegativeResponseCode	Cause of Occurrence
12	sub-functionNotSupported	The sub-function in request is not supported
13	incorrectMessageLengthOrInvalidFormat	The length of the message is wrong

Copyright 2024 © Knorr-Bremse AG. All rights reserved, including industrial property rights applications.
Knorr-Bremse AG retains any power of disposal, such as for copying and transferring.

5.2 (0x11) ECUReset Service

5.2.1 ECUReset Request

Below table defines the request message. The SuppressPosRspMsgIndicationBit (bit7) is also supported but not shown.

Byte	Parameter Name	Cvt	Value [Hex]	Mnemonic
#1	ECUReset Request SID	M	11	ER
#2	subFunction : resetType =	M	XX=	RT
	hardReset		01	

5.2.2 ECUReset Positive Response

Below table defines ECUReset positive response message flow example.

Byte	Parameter Name	Cvt	Value [Hex]	Mnemonic
#1	ECUReset Positive Response	M	51	ERPR
#2	resetType = hardRest	M	01	RT

5.2.3 Supported Negative Response Codes

NRC [Hex]	NegativeResponseCode	Cause of Occurrence
12	sub-functionNotSupported	The sub-function in request is not supported
13	incorrectMessageLengthOrInvalidFormat	The length of the message is wrong

5.3 (0x27) SecurityAccess Service

5.3.1 Security Access Request Seed

Below table defines the request message - request seed.

Byte	Parameter Name	Cvt	Value [Hex]	Mnemonic
#1	SecurityAcces Request SID	M	27	SA
#2	subFunction : securityAccessType =	M	XX=	SAT
	requestSeed		01	RSD
#3 . . #n	securityAccessDataRecord =	U	XX=	SECACCDR
	parameter#1 . . parameter#m		00 - FF . . 00 - FF	

securitySeed: Data value sent by server randomly and is used by client to calculate key to clear security access.

5.3.2 Security Access Send Key

Below table defines the request message – send key.

Copyright 2024 © Knorr-Bremse AG. All rights reserved, including industrial property rights applications.
Knorr-Bremse AG retains any power of disposal, such as for copying and transferring.

Byte	Parameter Name	Cvt	Value [Hex]	Mnemonic
#1	SecurityAcces Request SID	M	27	SA
#2	subFunction : securityAccessType =	M	XX=	SAT
	sendKey		02	SK
#3 . . #n	securityKey =	U	XX=	SECKEY
	key#1 (high byte) . . key#m (low byte)		00 - FF . . 00 - FF	

security Key: The “Key” parameter in the request message is the value generated in the tester by the following security algorithm corresponding to a specific “Seed” value.

```
#define Extended_tumbler1 (unsigned long) 9835;
#define Extended_tumbler2 (unsigned long) 6558;
unsigned long Extended_Session_Get_Key (unsigned long seed)
{
   return ((seed * Extended_tumbler1) + Extended_tumbler2);
}
```

5.3.3 Security Access Positive Response

Below table defines SecrityAccess positive response message flow example.

Byte	Parameter Name	Cvt	Value [Hex]	Mnemonic
#1	SecurityAccess Response SID	M	67	SAPR
#2	securityAccessType = requestSeed	M	01	SAT
#3 . . #n	seed#1 (high byte) . . seed#m (low byte)	C . . C	00 - FF . . 00 - FF	

5.3.4 Supported Negative Response Codes

NRC [Hex]	NegativeResponseCode	Cause of Occurrence
12	sub-functionNotSupported	The sub-function in request is not supported
13	incorrectMessageLengthOrInvalidFormat	The length of the message is wrong
24	requestSequenceError	The ‘sendKey’ as sub-function is received without first receiving a ‘requestSeed’ request message.
35	invalidKey	The key calculated by server and the key sent by client do not match.
36	exceededNumberOfAttempts	The attempt allowed for unlocking server exceeded.
37	requiredTimeDelayNotExpired	The request to unlock server is sent before a timer/delay (next attempt allowed after certain time periodic) is not expired.

Copyright 2024 © Knorr-Bremse AG. All rights reserved, including industrial property rights applications.
Knorr-Bremse AG retains any power of disposal, such as for copying and transferring.

5.4 (0x2E) WriteDataByIdentifier Service

5.4.1 WriteDataByIdentifier request

Below table defines the request message.

Byte	Parameter Name	Cvt	Value [Hex]	Mnemonic
#1	WriteDataByIdentifier Request SID	M	2E	WDBI
#2 #3	dataIdentifier =	M	XX=	DID
	byte#1 (MSB) byte#2 (LSB)		00 - FF 00 - FF	
#4 . . #m+3	dataRecord =	M	XX=	DREC
	data#1 . . data#m		00 - FF . . 00 - FF	

dataIdentifier: This parameter identifies the server data record that the client is requesting to write.

dataRecord: This parameter provides the data record associated with the dataIdentifier that the client is requesting to write to.

5.4.2 WriteDataByIdentifier Positive Response

Below table defines WriteDataByIdentifier positive response message flow example.

Byte	Parameter Name	Cvt	Value [Hex]	Mnemonic
#1	WriteDataByIdentifier Response SID	M	6E	WDBIPR
#2 #3	dataIdentifier = [byte#1 (MSB) byte#2 (LSB)]	M	00 - FF 00 - FF	DID

5.4.3 Supported Negative Response Codes

NRC [Hex]	NegativeResponseCode	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	The length of the message is wrong
31	requestOutOfRange	The request is not supported.
33	securityAccessDenied	The server is secure when a request for this service has been received.
72	generalProgrammingFailure	This return code shall be sent if the server detects an error when programming a memory location in the permanent memory device (e.g., Flash Memory) during the download of data.

Copyright 2024 © Knorr-Bremse AG. All rights reserved, including industrial property rights applications.
Knorr-Bremse AG retains any power of disposal, such as for copying and transferring.

6 Appendix – Example Logs

Timing	ID	Data	Explanation
2.435362	0x18DA2AF0	02 10 03 00 00 00 00 00	Entry ExtendedDiagnosticSession
2.439410	0x18DAF02A	06 50 03 00 32 01 F4 00	
2.456146	0x18DA2AF0	02 27 01 00 00 00 00 00	Security Unlock for ExtendedDiagnosticSession
2.459270	0x18DAF02A	06 67 01 A5 5B 2E C9 00	
2.475697	0x18DA2AF0	06 27 02 A6 0E 7D A1 00	
2.479305	0x18DAF02A	02 67 02 00 00 00 00 00	
2.497657	0x18DA2AF0	10 25 2E F1 85 01 07 E4	Write Fingerprint
2.498229	0x18DAF02A	30 08 14 00 00 00 00 00	
2.519053	…	21 08 1F 4B 42 44 00 FF	
2.539809	…	22 FF FF 00 00 00 00 00	
2.560588	…	23 00 00 00 00 00 00 00	
2.581400	…	24 00 00 00 00 00 00 00	
2.602204	…	25 00 00 00 00 00 00 00	
2.609296	0x18DAF02A	03 6E F1 85 00 00 00 00	
2.628131	0x18DA2AF0	14 03 2E FE B2 F1 87 00	Write Application-Data
2.628703	0x18DAF02A	30 08 14 00 00 00 00 00	
2.649367	0x18DA2AF0	21 1C 00 00 00 00 00 00	
2.670342	…	22 00 00 00 00 00 00 00	
2.691138	…	23 00 00 00 00 00 00 00	
2.711954	…	24 FF FF FF FF FF FF FF	
2.732714	…	25 FF F1 88 00 0C 00 00	
2.753609	…	26 00 00 00 00 00 00 00	
2.774789	…	27 00 00 00 F1 89 00 0C	
2.795952	…	28 00 00 00 00 00 00 00	
2.798240	0x18DAF02A	30 08 14 00 00 00 00 00	
2.819120	0x18DA2AF0	29 00 FF FF FF FF F1 91	
2.839892	…	2A 00 1C 00 00 00 00 00	
2.860680	…	2B 00 00 00 00 00 00 00	
2.881499	…	2C 00 00 00 FF FF FF FF	
2.902283	…	2D FF FF FF FF FF FF FF	
2.923051	…	2E FF FF F1 92 00 0C 4B	
2.943842	…	2F 32 37 38 35 35 33 00	
2.964778	…	20 00 00 00 FF F1 93 00	
2.965910	0x18DAF02A	30 08 14 00 00 00 00 00	
2.986629	0x18DA2AF0	21 0C 30 30 30 FF FF FF	
…	…	…	
…	…	…	
5.644703	…	20 FF FF FF FF 20 24 04	
5.648147	0x18DAF02A	30 08 14 00 00 00 00 00	
5.668959	0x18DA2AF0	21 24 11 07 03 FF 00 00	
5.689747	…	22 00 0C 00 01 F7 9F 00	
5.739314	0x18DAF02A	03 7F 2E 78 00 00 00 00	
5.742593	0x18DA2AF0	02 3E 80 00 00 00 00 00	
6.108251	0x18DA2AF0	02 3E 80 00 00 00 00 00	
6.179201	0x18DAF02A	03 6E FE B2 00 00 00 00	
7.735369	0x18DA2AF0	02 11 01 00 00 00 00 00	Reset ECU
7.739001	0x18DAF02A	02 51 01 00 00 00 00 00	

Copyright 2024 © Knorr-Bremse AG. All rights reserved, including industrial property rights applications.
Knorr-Bremse AG retains any power of disposal, such as for copying and transferring.